TP5 : Évolution « *Password Change* » et déploiement Docker

Objectif:

Vous devez reprendre et cloner le TP4 et :

- Ajouter une nouvelle fonctionnalité : **changement de mot de passe**
- Conserver toutes les règles de sécurité du TP4
- Conteneuriser l'application avec **Docker**
- Automatiser le ***build*** avec GitHub Actions

Base de départ

Le TP5 commence à partir du projet TP4

Le projet doit déjà contenir :

- Authentification sécurisée
- Master Key côté serveur pour chiffrer les mots de passe
- Tests JUnit
- Analyse SonarCloud
- Pipeline GitHub Actions

Vous devez:

- Cloner leur dépôt TP4
- Créer une nouvelle version Git

Commande :

- git clone https://github.com/votre-repo/tp4-auth.git
- cd tp4-auth

Nouvelle fonctionnalité

Il faut permettre à un utilisateur authentifié de changer son mot de passe.

Créer une branche :

- git checkout -b tp5-change-password

Le changement de mot de passe:

- Doit respecter toutes les règles suivantes
- L'utilisateur doit être authentifié
- L'ancien mot de passe doit être vérifié
- Le nouveau mot de passe doit respecter les règles de sécurité
- Le nouveau mot de passe doit être chiffré avec la Master Key

API à implémenter

Créer un nouvel endpoint REST.

Endpoint :

- PUT /api/auth/change-password

Exemple de requête JSON:

```
{
"email": "toto@example.com",
"oldPassword": "pwd1234",
"newPassword": "NewPassword123!",
"confirmPassword": "NewPassword123!"
}
```

Logique du changement de mot de passe

Le serveur doit effectuer les étapes suivantes :

1. Vérifier que l'utilisateur existe
2. Vérifier que l'ancien mot de passe est correct
3. Vérifier que newPassword et confirmPassword sont identiques
4. Vérifier la force du nouveau mot de passe
5. Chiffrer le nouveau mot de passe avec la Master Key
6. Mettre à jour la base de données
7. Si une règle n'est pas respectée, une exception doit être levée.

Sécurité du mot de passe

Le nouveau mot de passe doit respecter les mêmes règles que précédemment suivantes :

- Minimum 12 caractères
- Au moins une majuscule
- Au moins une minuscule
- Au moins un chiffre
- Au moins un caractère spécial

Le mot de passe doit ensuite être :

- chiffré avec la Master Key
- stocké dans la base de données

Tests JUnit

Les tests unitaires sont **obligatoires**.

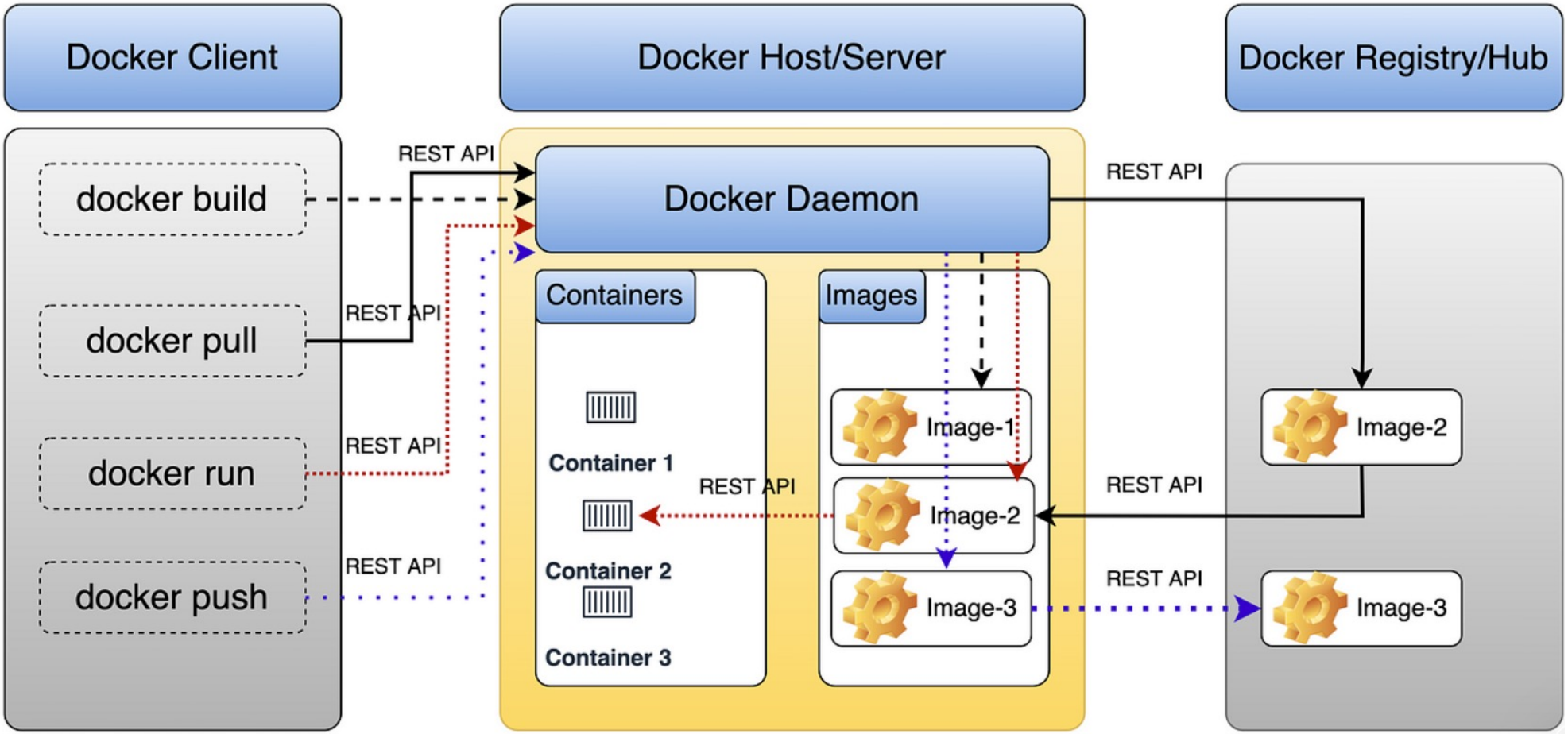
Cas de test minimum :

- Changement de mot de passe réussi
- Ancien mot de passe incorrect
- Confirmation différente
- Mot de passe trop faible
- Utilisateur inexistant

Objectif :

- Vérifier que la logique métier fonctionne correctement.

Rappel du cours



Objectif

- Rendre l'application déployable dans un conteneur Docker.
- Créer un fichier Dockerfile

Le conteneur doit :

- Copier le fichier jar
- Exposer le port 8080
- Démarrer l'application Spring Boot

Exemple de Dockerfile

```
FROM openjdk:17-jdk-slim

WORKDIR /app

COPY target/app.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java","-jar","/app/app.jar"]
```

Construction de l’image Docker

Construire l’image :

- docker build -t cdwfs-auth-app .

Lancer le conteneur :

- **docker run** -p 8080:8080
  -e APP_MASTER_KEY=test_master_key
  cdwfs-auth-app

L’application doit être accessible sur :

- http://localhost:8080

Pipeline CI/CD GitHub Actions

Le pipeline GitHub Actions doit maintenant :

- Build le projet
- Exécuter les tests
- Analyser le code avec SonarCloud
- Construire l’image Docker

Le pipeline doit échouer si :

- Un test échoue
- Le Quality Gate SonarCloud échoue
- Le build Docker échoue

Versionning obligatoire

```
git add .
git commit -m “Add change password feature”

git add .
git commit -m “Add Dockerfile”
```

```
git add .
git commit -m “Update GitHub Actions for Docker build”

git tag v5-tp5

git push origin main –tags
```

Savoir faire acquis

TP1 – Authentification dangereuse

- Comprendre le fonctionnement de base d'une authentification utilisateur dans une API REST.
- Implémenter une authentification simple avec Spring Boot, JUnit, JavaDoc et gestion d'exceptions.
- Identifier les risques de sécurité liés au stockage de mots de passe en clair.

TP2 – Authentification fragile

- Mettre en place des règles de sécurité pour les mots de passe (longueur, complexité).
- Implémenter le hachage des mots de passe côté serveur.
- Analyser la qualité du code et détecter les problèmes avec SonarCloud.

TP3 – Authentification forte

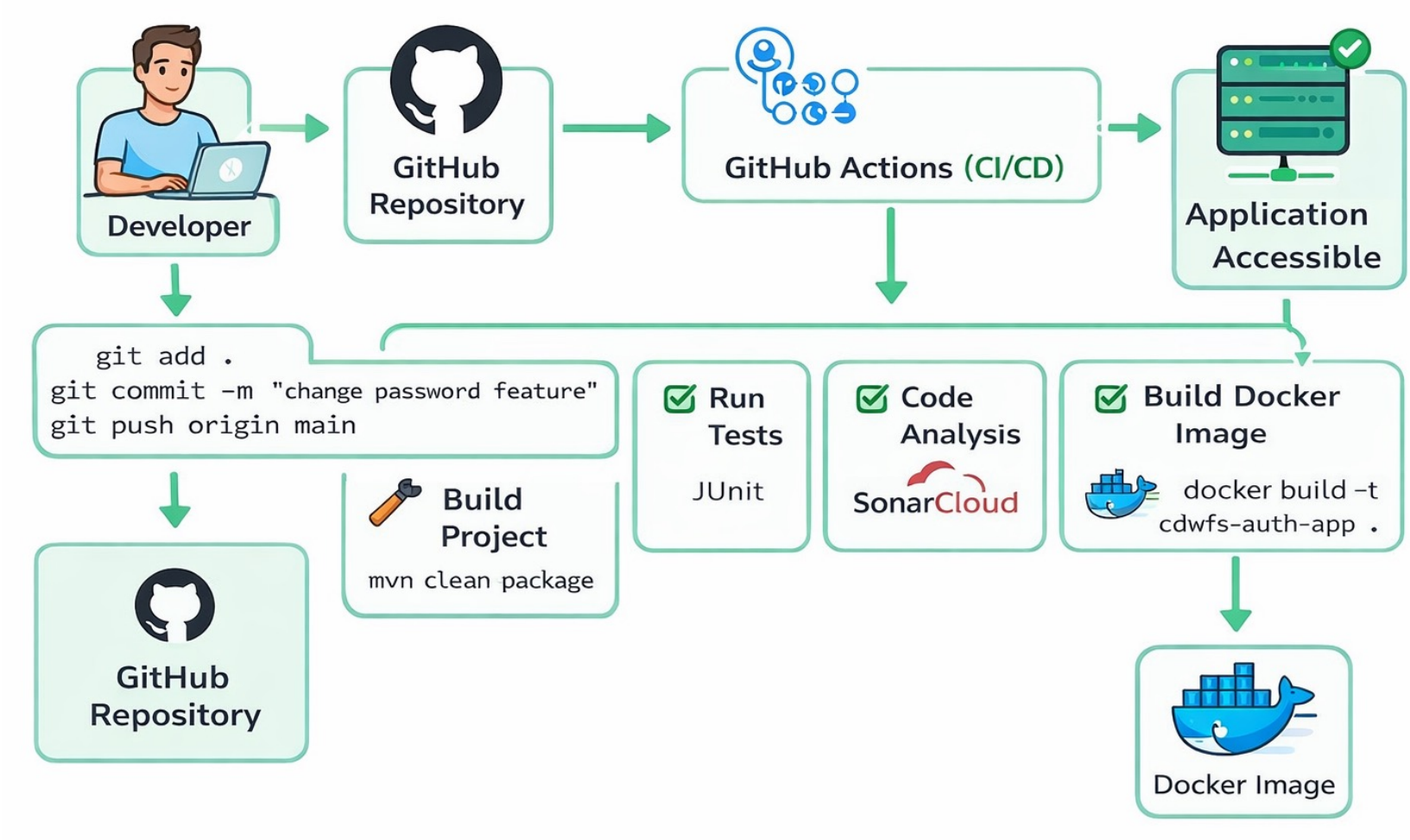
- Implémenter un protocole d'authentification sécurisé basé sur HMAC, nonce et timestamp.
- Comprendre les mécanismes de protection contre les attaques de rejeu.
- Développer et tester une logique d'authentification plus robuste.

TP4 – Industrialisation (Master Key + CI GitHub Actions)

- Mettre en place un chiffrement des mots de passe avec une Master Key côté serveur.
- Automatiser le build, les tests et l'analyse qualité avec GitHub Actions.
- Appliquer une démarche DevSecOps pour garantir la qualité et la sécurité du code.

TP5 – Évolution fonctionnelle et déploiement Docker

- Ajouter une nouvelle fonctionnalité sécurisée : changement de mot de passe.
- Conteneuriser une application Spring Boot avec Docker.
- Automatiser la construction de l'image Docker dans un pipeline CI/CD.


Developer
GitHub Repository
GitHub Actions (CI/CD)
Application Accessible
git add .
git commit -m "change password feature"
git push origin main
GitHub Repository
Build Project
mvn clean package
Run Tests
JUnit
Code Analysis
SonarCloud
Build Docker Image
docker build -t cdwfs-auth-app .
Docker Image

Fichier GitHub Actions: .github/workflows/ci.yml

Le TP5 garde exactement la même pipeline et on ajoute Docker.

Donc GitHub Actions devient maintenant :

- Build
- Tests
- SonarCloud
- Build Docker image

Le TP5 démontre **l'évolution naturelle d'une pipeline DevOps.**

```yaml
name: CI Build Test Sonar Docker

on:
  push:
    branches: ["main"]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:

      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Install JDK
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 17

      - name: Build project
        run: mvn clean package

      - name: Run tests
        run: mvn test

      - name: SonarCloud analysis
        env:
          SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
        run: mvn sonar:sonar

      - name: Build Docker image
        run: docker build -t cdwfs-auth-app .
```